Agenda de Contados com Dicionário

Mário Leite

Uma das estruturas de dados mais importantes do Python é o **dicionário** (*dict*). Ele permite armazenar pares chave ► valor, funcionando como uma tabela de acesso direto; isto o torna ideal para representar situações do mundo real como uma agenda de contatos, onde cada nome está associado a um telefone. Nesta postagem vamos construir uma agenda simples, orientada a objetos, utilizando uma classe e um dicionário interno para armazenar os dados.

Estrutura básica da classe

1. **ClsAgenda** possui um atributo principal: **DicContatos**, que é um dicionário onde:

- a chave é o nome do contato
- o valor é o telefone

2. **class ClsAgenda:**

```
def __init__(self):
 self.DicContatos = {}
```

Sempre que um objeto da agenda é criado, ele começa com um dicionário vazio.

Adicionando contatos

3. **AdicionarContato insere ou atualiza um contato:**

```
def AdicionarContato(self, nome, telefone):
     self.DicContatos[nome] = telefone
     print(f"Contato {nome} adicionado com sucesso!")
```

Em Python, se a chave já existir, o valor é automaticamente atualizado; ou seja, esse mesmo método serve tanto para inserir quanto para alterar.

Buscando um contato

```
def BuscarContato(self, nome):
     if nome in self.DicContatos:
        print(f"Telefone de {nome}: {self.DicContatos[nome]}")
     else:
        print(f"Contato {nome} não encontrado na agenda.")
```

Aqui vemos uma operação típica de dicionário: verificar se uma chave existe usando o operador in.

Listando todos os contatos

```
def ListarContatos(self):
    print("\nLista de Contatos:")
    for nome, telefone in self.DicContatos.items():
        print(f"{nome}: {telefone}")
    print("Fim da lista\n")
```

O método **items()** permite percorrer simultaneamente *chaves* e *valores*.

Programa principal (uso real)

```
Agenda = ClsAgenda()
Agenda.AdicionarContato("Alice", "123456789")
Agenda.AdicionarContato("Beto", "987654321")
Agenda.ListarContatos()
Agenda.BuscarContato("Alice")
Agenda.BuscarContato("Beto")
Agenda.AdicionarContato("Beto", "555555555")
Agenda.ListarContatos()
```

Observe que:

- "Beto" é adicionado duas vezes
- Na segunda vez, o telefone é substituído automaticamente

Os conceitos importantes envolvidos são os seguintes:

- Encapsulamento: o dicionário fica protegido dentro da classe
- Mapeamento chave-valor: essência dos dicionários
- CRUD básico:
    - **Create** ==> **AdicionarContato**
    - **Read** ==> **BuscarContato**
    - **Update** ==> **AdicionarContato** novamente
    - (Delete poderia ser incluído facilmente)

Vantagens de usar dicionário

Para uma agenda, o dicionário é ideal porque:

- busca em tempo praticamente O(1)
- não permite chaves duplicadas
- código limpo e legível
- modela diretamente a realidade

Possíveis extensões didáticas

Esse programa pode evoluir facilmente para:

- remover contato
- salvar em arquivo
- carregar da memória
- permitir vários telefones
- interface gráfica ou web

Este tipo de exemplo mostra como estruturas simples, quando bem escolhidas, resolvem problemas reais com pouquíssimo código. O **dicionário** não é apenas uma estrutura de dados; é uma forma de pensar soluções baseadas em associação direta. Para ensino de lógica e programação, uma agenda como esta é um dos melhores exemplos possíveis: concreta, útil e conceitualmente rica.
O programa" **AgendaDicionarioClasse**" é um exemplo bem interessante de aplicação; e a **figura 1** mostra a saída do programa.

```python
'''
AgendaDicionarioClasse.py
-------------------------------------------------------------------
Cria uma agenda usando Dicionário.
-------------------------------------------------------------------
'''
class ClsAgenda:
    def __init__(self):
        self.DicContatos = {}

    def AdicionarContato(self, nome, telefone):
        self.DicContatos[nome] = telefone
        print(f"Contato {nome} adicionado com sucesso!")

    def BuscarContato(self, nome):
        if(nome in self.DicContatos):
            print(f"Telefone de {nome}: {self.DicContatos[nome]}")
        else:
            print(f"Contato {nome} não encontrado na agenda.")

    def ListarContatos(self):
        print("\nLista de Contatos:")
        for nome, telefone in self.DicContatos.items():
            print(f"{nome}: {telefone}")
        print("Fim da lista\n")
#=====================================================================
#Programa principal
Agenda = ClsAgenda()  #cria uma instância da classe "ClsAgenda"
Agenda.AdicionarContato("Alice", "123456789")
Agenda.AdicionarContato("Beto", "987654321")
Agenda.ListarContatos()
Agenda.BuscarContato("Alice")
Agenda.BuscarContato("Beto")
Agenda.AdicionarContato("Beto", "555555555")
Agenda.ListarContatos()
#Fim do programa "AgendaDicionarioClasse" -------------------------
```

```
Run:  AgendaDicionarioClasse
C:\Users\Usuario\AppData\Local\Python\python310\Scripts\python.exe "D:\Livros\Livro11\
Contato Alice adicionado com sucesso!
Contato Beto adicionado com sucesso!

Lista de Contatos:
Alice: 123456789
Beto: 987654321
Fim da lista

Telefone de Alice: 123456789
Telefone de Beto: 987654321
Contato Beto adicionado com sucesso!

Lista de Contatos:
Alice: 123456789
Beto: 555555555
Fim da lista


Process finished with exit code 0
```

Figura 1 - Saída do programa "AgendaDicionarioClasse"